

EngineProject

DirectX 11 기반 미니 게임 엔진 구조 설명서

작성자 : Seokwon Park

브랜치 : dev

문서 버전 : 1.0

작성일 : 2026-09-09

목차

1. 엔진 개요
 2. 설계 목표 · 기술 스택 · 명명 규칙
3. 프로젝트 구조
 4. 디렉터리 레이아웃 · 모듈 의존 관계
5. Core 계층
 6. 타입 · 컨테이너 · 윈도우 · 타이머 · 로그
7. Math 라이브러리
 8. FVector · FMatrix · FQuat · FRotator · SIMD
9. Object System (리플렉션)
 10. UObject · UClass · 프로퍼티 매크로 · 팩토리 · 캐스팅
11. World / Actor / Component
 12. 스폰 · 틱 · 컴포넌트 계층
13. 렌더링 파이프라인
 14. FRenderer · 버퍼 · 셰이더 · 렌더 패킷 · 리소스 매니저
15. 카메라 시스템
 16. 뷰/투영 행렬 · 디프로젝션
17. 입력 시스템 & 피킹
 18. 키/마우스 상태 머신 · 레이캐스트 2단계 판정
19. 에디터 계층
 20. 패널 · 기즈모 · 아웃라인 · 그리드
21. 씬 직렬화
 22. JSON 포맷 · 저장/불러오기 흐름
23. 프레임 루프 전체 흐름
24. 확장 가이드
 25. 새 프리미티브 · 새 컴포넌트 · 새 패널 추가하기

1. 엔진 개요

EngineProject는 Direct3D 11 위에 언리얼 엔진의 아키텍처를 축소 재현한 학습용 게임 엔진입니다. 런타임과 에디터가 하나의 실행 파일에 통합되어 있으며, ImGui 기반 에디터 UI에서 오브젝트를 스폰·선택·조작하고 씬을 JSON으로 저장/불러올 수 있습니다.

1.1 설계 목표

- **UE 스타일 아키텍처 학습** — `UObject` / `UClass` / `AActor` / `UActorComponent` 계층을 직접 구현하여 리플렉션과 오브젝트 팩토리의 동작 원리를 이해합니다.
- **표준 라이브러리 최소 노출** — `TArray`, `TMap`, `TSharedPtr` 등 엔진 자체 별칭으로 감싸 향후 커스텀 컨테이너로 교체 가능하도록 합니다.
- **수학 라이브러리 자체 구현** — 벡터/행렬/쿼터니언을 직접 작성하고 SSE 인트린식으로 가속합니다.
- **에디터 우선** — 피킹, 기즈모, 아웃라인, 프로퍼티 편집을 갖춘 최소 에디터를 목표로 합니다.

1.2 기술 스택

항목	내용
언어 / 표준	C++ (MSVC, <code>std::format</code> 구조적 바인딩 사용 → C++20)
그래픽 API	Direct3D 11 (<code>d3d11.lib</code> , <code>d3dcompiler.lib</code>)
플랫폼	Win32 (<code>WinMain</code> 진입, 자체 <code>Window</code> 래퍼)
UI	Dear ImGui (Win32 + DX11 백엔드)
직렬화	nlohmann/json
SIMD	SSE (<code><immintrin.h></code> , <code>_mm128</code>)
스마트 포인터	COM 객체는 <code>Microsoft::WRL::ComPtr</code> , 엔진 리소스는 <code>TSharedPtr</code>

1.3 명명 규칙

접두사	의미	예시
U	<code>UObject</code> 파생 클래스	<code>UWorld</code> , <code>UPrimitiveComponent</code> , <code>UClass</code>
A	<code>AActor</code> 파생 (월드에서 배치되는 오브젝트)	<code>AActor</code> , <code>ACameraActor</code>
F	<code>UObject</code> 가 아닌 일반 구조체/클래스	<code>FRenderer</code> , <code>FTransform</code> , <code>FGizmo</code>
E	열거형	<code>EPrimitiveType</code> , <code>EGizmoMode</code>
I	인터페이스	<code>IEditorPanel</code>
T	템플릿 별칭	<code>TArray</code> , <code>TSharedPtr</code> , <code>TQueue</code>
b	bool 변수	<code>bIsRunning</code> , <code>bVisible</code>

2. 프로젝트 구조

2.1 디렉터리 레이아웃

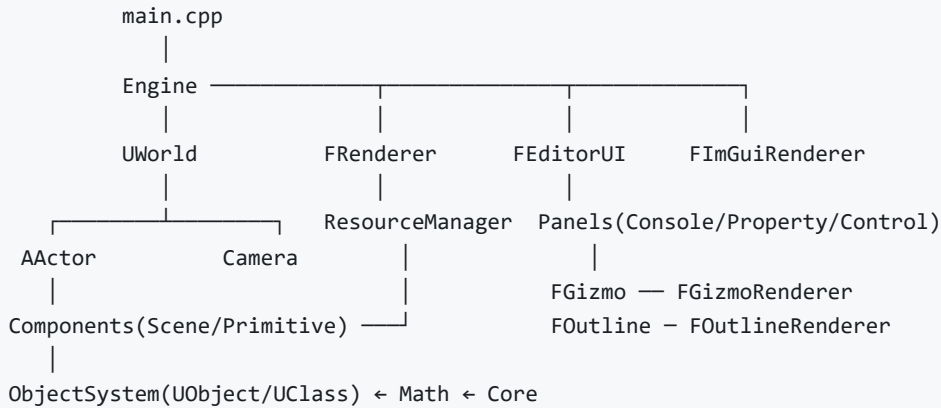
EngineProject/	
├ main.cpp	WinMain 진입점 - Engine 생성/Init/Run/Shutdown
├ EnginePCH.h	미리 컴파일된 헤더 (json, filesystem, 공통 타입)
├ ImGuiRenderer.*	ImGui 초기화 / 프레임 Begin·End
├ Texture2D.*	텍스처 리소스 래퍼
├	
├ Core/	플랫폼·기반 유틸
│ └ Types.h	int32/uint32 · TUniquePtr/TSharedPtr
│ └ Containers.h	TArray / TMap / TSet / TQueue
│ └ EngineString.h	FString 별칭
│ └ Window.*	Win32 윈도우 생성 · 메시지 펌프 · 리사이즈
│ └ EngineTimer.*	델타 타임 측정
│ └ EngineStatics.h	전역 카운터 (NextUUID, 할당 통계)
│ └ EngineLog.h	LOG(Verbosity, ...) 매크로
├	
├ Math/	자체 수학 라이브러리
│ └ Vector/Vector2/Vector4, Matrix, Quat, Rotator, Transform	
│ └ VectorRegister_SSE.h	SSE 인트린식 래퍼
├	
├ ObjectSystem/	리플렉션 · RTTI
│ └ Object.*	UObject, DECLARE_CLASS / PROPERTY 매크로
│ └ Class.*	UClass - 이름·부모·생성자·프로퍼티 테이블
│ └ Property.h	FProperty, EPropertyType
│ └ ObjectFactory.*	UClass로부터 인스턴스 생성 + UUID 부여
│ └ Casts.h	Cast / CastChecked
├	
├ GameFramework/Actor.*	AActor - 컴포넌트 소유 · BeginPlay/Tick
├ Component/	UActorComponent → USceneComponent → UPrimitiveComponent (+ CubeComponent, SphereComponent)
├ Camera/	ACameraActor, UCameraComponent
├ Collision/Ray.*	레이 · AABB/삼각형 교차 · 화면 투영 유틸
├ Input/	FInputSystem, EKeyCode, EMouseButton
├	
├ Engine/	
│ └ Engine.*	엔진 수명주기 + 메인 루프 소유
│ └ World.*	UWorld - 액터 목록 · 틱 · 피킹 · 씬 직렬화
│ └ ResourceManager.*	메시/셰이더 캐시 (싱글턴)
├	
├ Render/	
│ └ Renderer.*	D3D11 디바이스 · 스왑체인 · 드로우 호출
│ └ Buffer.*	FVertexBuffer / FIndexBuffer / FConstantBuffer
│ └ Shader.h	FShader (VS/PS/InputLayout)
│ └ Mesh.h	FMesh (VB + IB)
│ └ Vertex.*	FVertex + 입력 레이아웃 정의
│ └ RenderPacket.h	FRenderPacket (model, mesh, shader)
│ └ GeometryGenerator.*	프로시저럴 메시 생성 + FMeshData(AABB 포함)
├	
├ Editor/	
│ └ EditorUI.*	패널 컨테이너
│ └ EditorPanel.h	IEditorPanel 인터페이스

```

|   | ConsolePanel / PropertyPanel / ControlPanel
|   | Gizmo.* + GizmoRenderer.*      조작 로직 / 조작 렌더링 분리
|   | Outline.h + OutLineRenderer.* 선택 강조
|   | GridRenderer.*                월드 그리드
|
└ Shader/                          DefaultShader / GridShader / GizmoShader / OutlineShader (.hlsl)

```

2.2 모듈 의존 관계



화살표는 "사용" 방향입니다. **Core** 와 **Math** 는 최하위 계층으로 어떤 상위 모듈에도 의존하지 않습니다. **Editor** 는 **Engine** · **Render** 에 의존하지만 그 역은 성립하지 않도록 설계되어 있습니다(단, **EngineLog.h** 가 **Engine.h** 를 참조하는 예외가 있습니다).

3. Core 계층

3.1 타입과 컨테이너

엔진 전역에서 STL 타입을 직접 쓰지 않고 별칭을 통해 사용합니다. 이후 커스텀 자료구조로 교체할 때 호출부를 수정하지 않기 위한 층입니다.

```
// Core/Types.h
using int32 = int32_t; using uint32 = uint32_t;
template <typename T> using TUniquePtr = std::unique_ptr<T>;
template <typename T> using TSharedPtr = std::shared_ptr<T>;
template<typename T, typename... Args> TSharedPtr<T> MakeShared(Args&&... args);

// Core/Containers.h
template<typename T> using TArray = std::vector<T>;
template<typename T> using TQueue = std::queue<T>;
template<typename K, typename V> using TMap = std::unordered_map<K, V>;
template<typename T> using TSet = std::unordered_set<T>;
```

3.2 Window

`Window` 는 Win32 윈도우 클래스 등록, 핸들 생성, 메시지 펌프를 담당합니다. 윈도우 프로시저에서 받은 키보드-마우스 메시지는 `FInputSystem` 으로 전달되고, `WM_SIZE` 는 리사이즈 플래그로 기록되어 메인 루프에서 `CheckResized()` 로 소비됩니다.

```
MainWindow->ProcessMessage(bIsRunning); // PeekMessage 루프, WM_QUIT 시 bIsRunning=false
if (MainWindow->CheckResized())
{
    Renderer->Resize(MainWindow->GetWidth(), MainWindow->GetHeight());
    Camera->SetAspectRatio((float)Width / Height);
}
```

설계 포인트 — 리사이즈를 이벤트 콜백이 아니라 *폴링 플래그*로 처리합니다. 메시지 처리 중 스왑체인을 재생성하면 렌더링 중 리소스가 무효화될 수 있으므로, 프레임 경계에서 안전하게 처리하기 위함입니다.

3.3 EngineTimer

정적 클래스로, `Init()` 에서 고해상도 타이머 기준값을 잡고 매 프레임 `Tick()` 후 `GetDeltaTime()` 으로 프레임 간격(초)을 제공합니다. 델타 타임은 월드 틱, 에디터 틱, 카메라 이동 속도에 공통으로 쓰입니다.

3.4 로그

```
#define LOG(Verbosity, ...) \
    Engine::GetConsolePanel()->AddLog(ELogVerbosity::Verbosity, __VA_ARGS__);

LOG(Info, "Create Main Window...");
LOG(Error, "Failed To Create Main Window!");
```

로그는 곧바로 에디터의 콘솔 패널 버퍼에 쌓이며, `std::format` 스타일 가변 인자를 지원합니다(`LOG(Info, "{", Value)`).

3.5 EngineStatics

전역 카운터를 모아둔 구조체입니다. `NextUUID` 는 오브젝트 생성 시 순차 UUID를 발급하고, `TotalAllocationBytes` · `TotalAllocationCount` 는 `UObject::operator new/delete` 를 오버로드해 누적 집계합니다. 이 값들은 컨트롤 패널에 메모리 사용량으로 표시됩니다.

4. Math 라이브러리

4.1 구성

타입	역할
<code>FVector</code>	3D 벡터. <code>float V[3]</code> 와 <code>{X,Y,Z}</code> 를 union으로 겹쳐 배열 접근과 멤버 접근을 동시에 지원합니다.
<code>FVector2</code>	2D 벡터. 화면 좌표 계산(기즈모 픽킹, 스크린 투영)에 사용합니다.
<code>FVector4</code>	동차 좌표 / 색상(RGBA). 행렬 변환 결과 타입.
<code>FMatrix</code>	4×4 행렬. <code>Inverse()</code> , <code>TransformPosition()</code> , <code>TransformVector()</code> 제공.
<code>FQuat</code>	쿼터니언. 짐벌락 없는 회전 합성과 축-각 회전에 사용.
<code>FRotator</code>	오일러 각(Roll, Pitch, Yaw). 에디터 UI 노출용 표현.
<code>FTransform</code>	Location + Rotation + Scale 묶음. <code>GetWorldMatrix()</code> , <code>GetForward/Up/Right()</code> .

4.2 FTransform과 월드 행렬

```
struct FTransform
{
    FVector Location = FVector(0, 0, 0);
    FVector Scale = FVector(1, 1, 1);
    FRotator Rotation = FRotator(0, 0, 0);

    FMatrix GetWorldMatrix() const;    // S * R * T 순서로 합성
    FVector GetForward() const;        // 회전 적용된 기저 벡터
    FQuat GetOrientation() const;
};
```

`GetForward/Up/Right` 는 카메라 이동과 기즈모 축 방향 계산의 기반이 됩니다. 로컬 공간 기즈모 (`EGizmoSpace::Local`) 는 이 기저 벡터를, 월드 공간 기즈모는 전역 축을 사용합니다.

4.3 SIMD 가속

`Math/VectorRegister_SSE.h` 가 SSE 인트린식을 `VectorSIMD` 네임스페이스로 얹게 감쌉니다. x64가 아닌 플랫폼에서는 `float v[4]` 폴백 구조체로 컴파일됩니다.

```
using FVectorRegister = __m128;    // x64

namespace VectorSIMD
{
    inline FVectorRegister Load(const float* Ptr) { return _mm_loadu_ps(Ptr); }
    inline FVectorRegister Add(FVectorRegister A, FVectorRegister B) { return _mm_add_ps(A, B); }
    inline FVectorRegister Rsqrt(FVectorRegister V) { return _mm_rsqrt_ps(V); }
    inline FVectorRegister SetWZero(FVectorRegister A); // W 성분 마스크
}
```

주의 — `FVector` 는 `float V[3]` (12바이트)이므로 `_mm_load_ps` (16바이트 정렬 로드)로 직접 읽으면 버퍼 경계를 넘습니다. 정렬되지 않은 `Load ( _mm_loadu_ps )`를 쓰거나, 16바이트 정렬된 `FVector4` / `FMatrix` 경로에서만 SIMD를 적용해야 합니다.

5. Object System (리플렉션)

엔진의 심장부입니다. C++에는 런타임 리플렉션이 없으므로 매크로로 클래스 메타데이터를 수동 등록합니다.

5.1 UObject

```
class UObject
{
public:
    static UClass* StaticClass();
    virtual UClass* GetClass() const;

    template <typename T> bool IsA() { return IsA(T::StaticClass()); }
    bool IsA(UClass* Class);

    uint32 GetUUID() const;

    void* operator new(uint64 Size);      // 할당 바이트/횟수 누적 집계
    void operator delete(void* Ptr, uint64 Size);
private:
    uint32 UUID;
    uint32 InternalIndex;
};
extern TArray<UObject*> GUObjectArray;    // 전역 오브젝트 레지스트리
```

모든 `UObject` 는 고유 UUID를 가지며, 이 값이 싼 파일의 키로 사용됩니다. `operator new/delete` 오버로드 덕분에 엔진 오브젝트의 메모리 사용량을 별도 프로파일러 없이 추적할 수 있습니다.

5.2 DECLARE_CLASS 매크로

```
#define DECLARE_CLASS(Classname, SuperClassName) \
public: \
    using Super = SuperClassName; \
    using ThisClass = Classname; \
    static UClass* StaticClass() \
    { \
        static UClass c; \
        static bool bIsInit = false; \
        if (!bIsInit) \
        { \
            c.Name = #Classname; \
            c.Super = Super::StaticClass(); \
            c.Constructor = []() -> UObject* { return new Classname(); }; \
            Classname::RegisterProperties(&c); \
            bIsInit = true; \
        } \
        return &c; \
    } \
    virtual UClass* GetClass() const override { return StaticClass(); } \
private:
```

함수 지역 `static` 을 사용하므로 **최초 `StaticClass()` 호출 시점에 한 번만** 초기화됩니다(지연 초기화). 정적 초기화 순서 문제를 자연스럽게 회피하는 방식입니다. 람다로 저장한 `Constructor` 가 팩토리의 인스턴스 생성 수단이 됩니다.

5.3 프로퍼티 등록

```
class USceneComponent : public UActorComponent
{
    DECLARE_CLASS(USceneComponent, UActorComponent)

    REFLECT_START(ClassName)
        PROPERTY(bVisible)          // 이름 + 타입 + offsetof 를 UClass에 등록
    REFLECT_END()
    ...
};

// 확장 결과
InClass->AddProperty<decltype(ThisClass::bVisible)>("bVisible", offsetof(ThisClass, bVisible));
```

```
struct FProperty { FString Name; EPropertyType Type; size_t Offset; size_t Size; };
enum class EPropertyType { Unknown, Float, Int, String, Bool, Vector };
```

등록된 프로퍼티는 `UClass::GetProperties()` 로 순회할 수 있으며, 이름·오프셋·크기를 알기 때문에 프로퍼티 패널의 자동 UI 생성이나 `CopyProperties()` 같은 범용 복사에 활용됩니다.

```
inline void CopyProperties(UObject* Src, UObject* Dst, UClass* FromClass)
{
    for (UClass* c = FromClass; c; c = c->Super)          // 상속 체인 전체 순회
        for (const FProperty& p : c->Properties)
            memcpy((char*)Dst + p.Offset, (char*)Src + p.Offset, p.Size);
}
```

`PROPERTY` 로 등록하려는 타입은 `Property.h` 의 `DEFINE_PROPERTY_TYPE` 에 반드시 있어야 합니다. 없으면 `GetPropertyType<T>()` 의 `static_assert` 가 컴파일 타임에 걸립니다 — 의도된 안전장치입니다.

5.4 UClass와 타입 검사

```
class UClass : public UObject
{
public:
    FString          Name;
    UClass*          Super      = nullptr;
    ClassConstructor Constructor = nullptr;    // UObject* (*)()
    TArray<FProperty> Properties;
    UObject*         DefaultObject = nullptr; // CDO

    bool    IsChildOf(UClass* BaseClass) const;    // Super 체인을 거슬러 비교
    UObject* GetDefaultObject();                  // 최초 요청 시 생성 후 캐시
};
```

`IsChildOf` 는 `Super` 포인터 체인을 따라 올라가며 동일 `UClass` 주소를 찾는 방식으로, `dynamic_cast` 없이 상속 판정을 수행합니다. `Cast<T>` 는 실패 시 `nullptr`, `CastChecked<T>` 는 실패를 계약 위반으로 간주합니다.

5.5 오브젝트 팩토리

```
UObject* FObjectFactory::ConstructObject(UClass* Class);    // Class->Constructor() 호출 + UUID 발급

template <typename T>
static T* ConstructObject() { return CastChecked<T>(ConstructObject(T::StaticClass())); }
```

모든 엔진 오브젝트는 `new` 를 직접 쓰지 않고 팩토리를 통해 만듭니다. UUID 발급과 전역 등록을 한 곳으로 모으기 위함이며, 씬 로드 시 클래스 이름 문자열만으로 인스턴스를 만들 수 있는 근거가 됩니다.

6. World / Actor / Component

6.1 계층 구조

```
UObject
├─ UClass
├─ UWorld          썬 전체 - 액터 목록, 틱, 피킹, 직렬화
├─ AActor          월드에 배치되는 개체 (RootComponent 소유)
│   └─ ACameraActor  카메라 컴포넌트를 루트로 갖는 액터
└─ UActorComponent BeginPlay / TickComponent
    └─ USceneComponent FTransform · bVisible
        ├─ UPrimitiveComponent 메시 + 셰이더 + 렌더 큐 제출
        │   └─ UCubeComponent
        │       └─ USphereComponent
        └─ UCameraComponent 뷰/투영 행렬 · 디프로젝션
```

6.2 UWorld

```
class UWorld : public UObject
{
public:
    bool Init();
    AActor* SpawnActor(UClass* Class, const FTransform* Transform);
    template <class T> T* SpawnActor(const FTransform* Transform);

    void Tick(float DeltaTime);
    void GatherRenderPackets(TQueue<FRenderPacket>& RenderQueue);
    UPrimitiveComponent* GetPickingPrimitive(uint32 ScreenW, uint32 ScreenH);

    void ClearScene();
    bool NewScene (const FString& Path);
    bool SaveScene(const FString& Path);
    bool LoadScene(const FString& Path);

private:
    TArray<AActor*> Actors;
    TQueue<AActor*> BeginPlayList; // 지연 BeginPlay 큐
    TArray<UPrimitiveComponent*> PrimitiveComponents; // 렌더/피킹 평면 목록
    ACameraActor* MainCamera = nullptr;
};
```

스폰 절차

1. Class 가 AActor 의 자식인지 IsChildOf 로 검증 — 아니면 nullptr 반환
2. FObjectFactory::ConstructObject(Class) 로 인스턴스 생성
3. NewActor->World = this (UWorld 가 AActor 의 friend이므로 private 접근 가능)
4. 루트 컴포넌트가 없으면 기본 USceneComponent 를 만들어 전달받은 트랜스폼 적용
5. Actors 에 추가하고 BeginPlayList 에 넣어 다음 틱에 BeginPlay 호출 예약

틱 순서

```
void UWorld::Tick(float DeltaTime)
{
    while (!BeginPlayList.empty())           // 1) 신규 액터 BeginPlay 먼저 소진
    {
        BeginPlayList.front()->BeginPlay();
        BeginPlayList.pop();
    }
    for (AActor* Actor : Actors)             // 2) 전체 액터 Tick
        Actor->Tick(DeltaTime);
}
```

왜 큐를 쓰나 — 액터가 **Tick** 도중 다른 액터를 스폰해도 **BeginPlay** 가 즉시 재귀 호출되지 않고 다음 프레임으로 미뤄집니다. 순회 중 컨테이너 변경으로 인한 반복자 무효화도 함께 방지됩니다.

6.3 AActor

```
class AActor : public UObject
{
    DECLARE_CLASS(AActor, UObject)
public:
    virtual void BeginPlay();
    virtual void Tick(float DeltaTime);

    UPrimitiveComponent* GetPrimitiveComponent() const;           // RootComponent 다운캐스트
    void AddPrimitiveComponent(EPPrimitiveType Type, FTransform Transform);
    void SetRootComponent(USceneComponent* SceneComponent);

    friend class UWorld;
protected:
    TArray<UActorComponent*> Components;
private:
    UWorld* World = nullptr;
    USceneComponent* RootComponent = nullptr;
};
```

AddPrimitiveComponent 는 프리미티브 타입에 맞는 메시 데이터를 **FGeometryGenerator** / **FResourceManager** 에서 얻어 컴포넌트에 세팅하고, 월드의 **PrimitiveComponents** 목록에도 등록해 렌더링과 피킹 대상으로 만듭니다.

6.4 UPrimitiveComponent

```
class UPrimitiveComponent : public USceneComponent
{
public:
    void SetMeshData(const FMeshData& data);                // CPU 사본 - 피킹·AABB용
    void SetMeshShader(FMesh* InMesh, FShader* InShader);    // GPU 리소스 참조
    void SubmitToRenderQueue(TQueue<FRenderPacket>& RenderQueue);

    void          SetType(EPrimitiveType Ptype);
    EPrimitiveType GetType() const;
protected:
    EPrimitiveType Type = EPrimitiveType::Cube;
    FShader*       Shader = nullptr;    // 소유하지 않음 (ResourceManager 소유)
    FMesh*         Mesh   = nullptr;
    FMeshData      MeshData;           // CPU 정점/인덱스 사본
};
```

메시 데이터가 두 벌인 이유 — `FMesh` 는 GPU 버퍼라 CPU에서 읽을 수 없습니다. 레이-삼각형 피킹과 AABB 계산에는 정점 배열이 필요하므로 `FMeshData` CPU 사본을 함께 보관합니다. 대신 컴포넌트마다 정점 배열이 복제되므로, 개체 수가 늘면 `FMeshData` 도 `TSharedPtr` 로 공유하는 편이 좋습니다.

7. 렌더링 파이프라인

7.1 FRenderer의 역할

D3D11 디바이스·스왑체인·상태 객체를 소유하고, 리소스 생성 팩토리와 드로우 호출 인터페이스를 함께 제공합니다.

그룹	주요 함수
초기화	<code>Init</code> , <code>CreateDeviceAndSwapChain</code> , <code>CreateFrameBuffer</code> , <code>CreateRasterizerState</code> , <code>CreateDepthStencilBufferAndState</code>
리소스 생성	<code>CreateShader</code> , <code>CreateMesh</code> , <code>CreateVertexBuffer</code> , <code>CreateIndexBuffer</code> , <code>CreateConstantBuffer</code>
바인딩	<code>BindShader</code> , <code>BindMesh</code> , <code>BindVertexBuffer</code> , <code>BindIndexBuffer</code> , <code>BindConstantBuffer</code>
상태	<code>SetPrimitiveTopology</code> , <code>SetDepthStencilEnabled</code>
드로우	<code>Draw</code> , <code>DrawIndexed</code> , <code>RenderAll</code>
프레임	<code>BeginFrame</code> (클리어), <code>EndFrame</code> (Present), <code>Resize</code> , <code>Shutdown</code>

7.2 정점 포맷

```
struct FVertex
{
    FVector Position;    // 12 bytes
    FVector4 Color;      // 16 bytes
    static std::vector<D3D11_INPUT_ELEMENT_DESC> GetLayout()
    {
        return {
            { "POSITION", 0, DXGI_FORMAT_R32G32B32_FLOAT,    0, 0, D3D11_INPUT_PER_VERTEX_DATA, 0 },
            { "COLOR",    0, DXGI_FORMAT_R32G32B32A32_FLOAT, 0, 12, D3D11_INPUT_PER_VERTEX_DATA, 0 },
        };
    }
};
```

레이아웃 오프셋 주의 — COLOR의 오프셋 12는 `FVector`가 12바이트(`float V[3]`)일 때만 맞습니다. `FVector`를 16바이트로 되돌리면 이 값도 함께 바뀌어야 하므로, 상수 대신 `offsetof(FVertex, Color)`를 쓰는 편이 안전합니다. 같은 레이아웃이 `Engine::Init`에도 하드코딩되어 있어 `FVertex::GetLayout()`으로 일원화하면 중복이 사라집니다.

7.3 렌더 패킷과 수집-제출 구조

```
struct FRenderPacket
{
    FMatrix  model;    // 월드 행렬
    FMesh*   mesh;     // VB + IB
    FShader* shader;   // VS + PS + InputLayout
};
```

월드는 그리기 명령을 직접 호출하지 않고 *무엇을 그릴지*만 큐에 담습니다. 렌더러는 큐를 받아 실제 드로우를 수행합니다.

```
// 수집 (UWorld)
void UWorld::GatherRenderPackets(TQueue<FRenderPacket>& RenderQueue)
{
    for (UPrimitiveComponent* Primitive : PrimitiveComponents)
        if (Primitive)
            Primitive->SubmitToRenderQueue(RenderQueue);
}

// 소비 (FRenderer)
void FRenderer::RenderAll(TQueue<FRenderPacket>& InQueue, FMatrix VP);
// 각 패킷마다: MVP = model * VP → 상수 버퍼 갱신 → 셰이더/메시 바인딩 → DrawIndexed
```

이 구조의 이점 — 게임플레이 로직과 그래픽 API가 `FRenderPacket`이라는 데이터 경계로 분리됩니다. 나중에 정렬(불투명/반투명), 인스턴싱, 컬링, 멀티스레드 커맨드 기록을 추가할 때 큐 소비부만 고치면 됩니다.

7.4 버퍼 래퍼

```
class FVertexBuffer { uint32 Stride; ComPtr<ID3D11Buffer> Buffer; };
class FIndexBuffer { uint32 IndexCount; ComPtr<ID3D11Buffer> Buffer; };
class FConstantBuffer { void UpdateData(const void* InData); /* Map/Unmap */ };

struct FMesh { TSharedPtr<FVertexBuffer> VertexBuffer; TSharedPtr<FIndexBuffer> IndexBuffer; };
```

Stride와 IndexCount를 버퍼 자신이 들고 있으므로 드로우 시점에 호출부가 크기 정보를 따로 넘기지 않아도 됩니다.

7.5 셰이더 바인딩 플래그

```
enum EShaderBindFlagBits : uint32
{
    None = 0, Vertex = 1, Geometry = 1 << 2, Domain = 1 << 3,
    Hull = 1 << 4, Pixel = 1 << 5, Compute = 1 << 6
};

Renderer->BindConstantBuffer(0, CB, (EShaderBindFlagBits)(Vertex | Pixel));
```

`Vertex = 1` 다음이 `1 << 2 (= 4)`라서 비트 `2 (= 1 << 1)`가 비어 있습니다. 동작에 문제는 없지만 `1 << 0, 1 << 1, 1 << 2 ...`로 정리하면 일관됩니다.

7.6 ResourceManager

```
class FResourceManager    // 싱글턴 (복사·대입 금지)
{
public:
    static FResourceManager& GetInstance();
    void      Init(FRenderer* renderer);
    FMesh*    GetMesh  (FString InName);    // 없으면 생성 후 캐시, 있으면 재사용
    FShader*  GetShader(FString InName);
private:
    TMap<FString, TSharedPtr<FMesh>>    MeshMap;
    TMap<FString, TSharedPtr<FShader>>    ShaderMap;
    TMap<FString, TSharedPtr<FVertexBuffer>>    VertexBufferMap;
    TMap<FString, TSharedPtr<FIndexBuffer>>    IndexBufferMap;
};
```

키는 메시 이름("Cube", "Sphere") 또는 셰이더 파일 이름입니다. 같은 큐브를 100개 스폰해도 GPU 버퍼는 한 벌만 존재하며, 컴포넌트는 원시 포인터로 참조만 합니다(소유권은 매니저에 있음).

7.7 셰이더 목록

파일	용도
DefaultShader.hls1	일반 프리미티브 — MVP 변환 + 정점 색상 출력
GridShader.hls1	월드 그리드. 카메라 위치를 받아 무한 격자 느낌 연출
GizmoShader.hls1	기즈모 축. 깊이 테스트를 끄고 항상 위에 그림
OutlineShader.hls1	선택 강조. 스케일을 살짝 키운 실루엣

8. 카메라 시스템

8.1 UCameraComponent

```
class UCameraComponent : public USceneComponent
{
public:
    float FOV          = 60.0f;
    float AspectRatio   = 16.0f / 9.0f;
    float NearClipPlane = 0.1f;
    float FarClipPlane  = 10000.0f;
    bool  bIsOrthogonal = false;
    float OrthoWidth    = 16.0f;

    FMatrix GetViewMatrix() const;
    FMatrix GetPerspectiveMatrix() const;
    FMatrix GetOrthogonalMatrix() const;
    FMatrix GetViewProjectionMatrix() const;    // 투영 모드에 따라 분기

    FRay DeProjection(int32 MouseX, int32 MouseY, uint32 ScreenW, uint32 ScreenH);
private:
    const float CameraSpeed      = 10.0f;
    const float MouseSensitivity = 0.05f;
    const float WheelSpeed       = 0.1f;
};
```

뷰 행렬은 컴포넌트의 `FTransform` 에서 파생됩니다(`GetForward/Up` 사용). 카메라도 하나의 `USceneComponent` 이므로 트랜스폼 편집·직렬화 경로를 프리미티브와 공유합니다.

8.2 카메라 조작

`ACameraActor::Tick` 에서 입력을 읽어 트랜스폼을 갱신합니다. 우클릭 드래그로 회전(마우스 델타 × 감도 → Pitch/Yaw), WASD로 전후좌우 이동(`CameraSpeed × DeltaTime`), 휠로 이동 속도 또는 전진량을 조절하는 표준 FPS 스타일 플라이 카메라입니다.

8.3 디프로젝션 (스크린 → 월드 레이)

```
FRay ray = Camera->DeProjection(MouseX, MouseY, ScreenW, ScreenH);
```

1. 픽셀 좌표를 NDC로 변환 — $x = 2 \cdot \text{MouseX} / W - 1$, $y = 1 - 2 \cdot \text{MouseY} / H$
2. 투영 행렬의 역변환으로 뷰 공간 방향 복원
3. 뷰 행렬의 역변환으로 월드 공간 원점(카메라 위치)과 방향 산출
4. 방향 정규화 후 `FRay{ Origin, Direction }` 반환

이 레이 하나가 오브젝트 피킹과 기즈모 드래그 양쪽의 입력으로 재사용됩니다.

9. 입력 시스템 & 피킹

9.1 FInputSystem

전역 정적 클래스입니다(`private` 생성자 `= delete`). 현재 프레임과 이전 프레임의 상태 배열을 함께 보관해 "눌림/떼짐" 엣지를 판정합니다.

```
static bool IsKeyDown    (EKeyCode);    // 현재 눌러 있음        (지속)
static bool IsKeyPressed (EKeyCode);    // 이번 프레임에 눌림    (엣지)
static bool IsKeyReleased(EKeyCode);    // 이번 프레임에 떼짐    (엣지)
// 마우스도 동일한 3종 + 위치/델타/휠

static void UpdateInputStates();        // 프레임 끝에서 현재 → 이전 복사

static inline bool bKeyStates[256], bPrevKeyStates[256];
static inline bool bMouseStates[5],  bPrevMouseStates[5];
static inline int32 MouseX, MouseY, DeltaX, DeltaY, WheelDelta;
```

호출 순서가 중요 — `UpdateInputStates()` 는 그 프레임의 모든 입력 질의가 끝난 *뒤에* 호출되어야 합니다. 메인 루프에서는 기즈모 업데이트와 피킹 처리 후, 렌더링 직전에 호출합니다. 순서가 바뀌면 `IsPressed` 가 한 프레임도 `true` 가 되지 않습니다.

`GetWheelDelta()` 는 값을 반환하면서 `0` 으로 초기화하는 소비형 게터입니다. 프레임 내에서 두 곳이 호출하면 뒤쪽은 항상 0을 받으므로 주의가 필요합니다.

9.2 레이 유틸리티

```
struct FRay { FVector Origin; FVector Direction; FVector PointAt(float t) const; };

bool RayIntersectsAABB    (const FRay&, const FVector& Min, const FVector& Max, float& OutT);
bool RayIntersectsTriangle(const FRay&, const FVector& v1, const FVector& v2, const FVector& v3,
float& OutT);
bool RayIntersectsPlane   (const FRay&, const FVector& Point, const FVector& Normal, float& OutT);

FVector2 WorldToScreen    (const FVector& WorldPos, const FMatrix& ViewProj, int W, int H);
float    DistanceToSegment(const FVector2& P, const FVector2& A, const FVector2& B);
```

9.3 피킹 알고리즘 — 2단계 판정

```
UPrimitiveComponent* UWorld::GetPickingPrimitive(uint32 ScreenW, uint32 ScreenH)
{
    FRay ray = MainCamera->GetCameraComponent()->DeProjection(MouseX, MouseY, ScreenW, ScreenH);
    float minT = FLT_MAX;
    UPrimitiveComponent* Picked = nullptr;

    for (UPrimitiveComponent* Primitive : PrimitiveComponents)
    {
        // ① 레이를 오브젝트 로컬 공간으로 역변환 (메시를 월드로 변환하지 않음)
        FMatrix invWorld = Primitive->GetWorldMatrix().Inverse();
        FRay LocalRay { invWorld.TransformPosition(ray.Origin),
                        invWorld.TransformVector (ray.Direction) };

        // ② Broad Phase - 로컬 AABB 테스트로 빠르게 기각
        const FMeshData& mesh = Primitive->GetMeshData();
        FVector BoxMin, BoxMax; mesh.GetAABB(BoxMin, BoxMax);
        float rayT;
        if (!RayIntersectsAABB(LocalRay, BoxMin, BoxMax, rayT)) continue;
        if (rayT > minT) continue; // 이미 더 가까운 후보가 있음

        // ③ Narrow Phase - 윌러-트럼보르 삼각형 교차
        for (uint32 i = 0; i + 2 < mesh.Indices.size(); i += 3)
        {
            /* 삼각형 3점 추출 후 RayIntersectsTriangle */
            if (rayT < minT) { Picked = Primitive; minT = rayT; }
        }
    }
    return Picked;
}
```

왜 레이를 로컬로 보내나 — 메시 정점 전체를 월드로 변환하면 정점 수만큼 행렬 곱이 발생합니다. 반대로 레이(원점+방향, 2개 벡터)를 역행렬로 한 번 변환하면 오브젝트당 역행렬 1회 + 벡터 변환 2회로 끝납니다. 정점이 많을수록 이득이 커집니다.

주의 두 가지 — (1) 로컬 레이의 `Direction` 은 비균등 스케일에서 길이가 바뀌므로, 반환되는 `t` 는 로컬 공간 기준입니다. 서로 다른 스케일의 오브젝트끼리 `minT` 를 직접 비교하면 순서가 뒤집힐 수 있습니다. 월드 공간 거리로 환산해 비교하면 정확해집니다. (2) 매 프레임 `Inverse()` 를 호출하므로, 트랜스폼이 바뀔 때만 역행렬을 캐시하면 비용을 줄일 수 있습니다.

10. 에디터 계층

10.1 패널 시스템

```
class IEditorPanel
{
public:
    virtual bool Init() = 0;
    virtual void Tick(float DeltaTime) = 0;
    virtual void OnRender() = 0;
};

class FEditorUI
{
public:
    template <typename T> T* AddEditorPanel()
    {
        TUniquePtr<T> newPanel = MakeUnique<T>();
        T* Ret = newPanel.get();
        Ret->Init();
        Panels.push_back(std::move(newPanel));
        return Ret; // 소유권은 컨테이너, 호출부는 관찰 포인터
    }
private:
    TArray<TUniquePtr<IEditorPanel>> Panels;
};
```

`FEditorUI::Tick / OnRender` 는 등록된 모든 패널에 순서대로 위임합니다. 패널 추가는 `AddEditorPanel<T>()` 한 줄이면 끝납니다.

패널	기능
<code>FConsolePanel</code>	<code>LOG</code> 매크로가 쌓은 로그 출력. Verbosity(Info/Warning/Error) 필터링
<code>FControlPanel</code>	FPS·델타타임 표시, 프리미티브 스폰(Sphere/Cube/Cone/Plane), 기즈모 모드·공간 전환, 씬 New/Save/Load
<code>FPropertyPanel</code>	선택된 <code>USceneComponent</code> 의 Location/Rotation/Scale 편집

10.2 씬 클리어 콜백

```
ControlPanel->SetSceneClearCallback([&]() {
    Gizmo        ->SetTarget(nullptr);
    Outline       ->SetTarget(nullptr);
    PropertyPanel->SetTarget(nullptr);
});
```

씬을 비울 때 액터가 `delete` 되므로, 그 액터를 가리키던 기즈모·아웃라인·프로퍼티 패널의 타겟을 반드시 함께 비워야 합니다. 콜백으로 묶어두면 `UWorld` 가 에디터를 몰라도(역방향 의존 없이) 정리가 이뤄집니다 — 댕글링 포인터 방지 장치입니다.

10.3 기즈모 — 로직과 렌더링 분리

`FGizmo` 는 상태와 조작 로직만, `FGizmoRenderer` 는 그리기만 담당합니다.

```
enum class EGizmoMode { Location, Rotation, Scale };
enum class EGizmoSpace { Local, World };

class FGizmo
{
public:
    void Update(const FRay& MouseRay, const FVector2& MousePos, const FMatrix& ViewProj,
               int ScreenW, int ScreenH, bool bMouseDown, UCameraComponent* Camera);

    int PickAxis(const FVector2& MousePos, const FMatrix& ViewProj, int W, int H);
    int PickLinearAxis (...); // 이동/스케일 - 축 선분과 마우스의 화면 거리
    int PickRotationAxis(...); // 회전 - 링과 마우스의 화면 거리

    void BeginDrag(int Axis, const FRay&, const FVector2&);
    void UpdateDrag(const FRay&, const FVector2&);
    void EndDrag();

    bool IsUsing() const { return DraggingAxis >= 0; }
    int GetHoveredAxis() const;
private:
    EGizmoMode Mode = EGizmoMode::Location;
    EGizmoSpace Space = EGizmoSpace::Local;
    USceneComponent* Target = nullptr;

    int HoveredAxis = -1, DraggingAxis = -1;
    FVector DragStartPoint, DragStartLocation, DragAxisDirection, DragPlaneNormal;
    FRotator DragStartRotation;
    FVector DragStartScale;
    float DragStartAngle;
};
```

드래그 동작 원리

- **축 선택** — 각 축의 3D 끝점을 `WorldToScreen` 으로 투영하고, `DistanceToSegment` 로 마우스와의 픽셀 거리를 재 임계값 안쪽이면 호버로 판정합니다. 3D 충돌체 없이 2D 거리만으로 판정하므로 구현이 간결하고, 화면상 겹친 축도 자연스럽게 구분됩니다.
- **이동/스케일** — 드래그 축을 포함하고 카메라를 향하는 평면(`DragPlaneNormal`)을 만들고, `RayIntersectsPlane` 으로 마우스 레이와의 교점을 구합니다. 시작 교점 대비 이동량을 축 방향으로 투영해 델타를 얻습니다.
- **회전** — `ComputeAngleOnPlane` 이 축에 수직인 평면 위에서 교점의 각도를 구합니다. 드래그 시작 각도와 차이가 회전량이 됩니다.
- **Local vs World** — `GetAxisDirection(Axis)` 이 `Space` 에 따라 타깃의 기저 벡터 또는 전역 축을 반환합니다.

`GetRenderLocation()` 과 `GetCameraLocation()` 은 카메라 거리에 비례해 기즈모 크기를 보정(화면상 일정 크기 유지)하는 데 쓰입니다.

10.4 아웃라인

```
class FOutline
{
public:
    void SetTarget(UPrimitiveComponent* InTarget);
    FMesh*      GetMesh()          const { return Target->GetMesh(); }
    const FMatrix& GetWorldMatrix() const { return Target->GetTransform()->GetWorldMatrix(); }
    const FVector& GetTargetScale() const { return Target->GetTransform()->Scale; }
private:
    UPrimitiveComponent* Target = nullptr;
};
```

`FOutlineRenderer` 가 타깃 메시를 약간 확대해 단색으로 그려 실루엣을 만듭니다.

`GetWorldMatrix()` 가 `const FMatrix&` 를 반환하는데 `FTransform::GetWorldMatrix()` 는 값(`FMatrix`)을 반환합니다. 임시 객체에 대한 참조를 돌려주는 형태이므로 반환 타입을 값(`FMatrix`)으로 바꾸는 것이 안전합니다. `GetTargetScale()` 도 `GetTransform()` 이 포인터를 반환하므로 수명 자체는 유효하지만, 일관성을 위해 함께 점검합니다.

10.5 그리드

`FGridRenderer::OnRender(VP, CameraLocation)` 이 뷰·투영 행렬과 카메라 위치를 받아 월드 격자를 그립니다. 카메라 위치를 넘기는 이유는 셰이더에서 거리 기반 페이드나 그리드 스냅 오프셋을 적용하기 위해서입니다.

11. 씬 직렬화

11.1 파일 포맷

씬은 `Scene/<이름>.Scene` 경로에 JSON으로 저장됩니다(디렉터리는 `std::filesystem::create_directories` 로 자동 생성).

```
{
  "Version": 1,
  "NextUUID": 7,
  "Primitives": {
    "1": {
      "Location": [0.0, 0.0, 0.0],
      "Rotation": [0.0, 0.0, 0.0],
      "Scale": [1.0, 1.0, 1.0],
      "Type": "Camera"
    },
    "2": {
      "Location": [3.0, 0.0, 1.5],
      "Rotation": [0.0, 45.0, 0.0],
      "Scale": [1.0, 1.0, 1.0],
      "Type": "Cube"
    }
  }
}
```

필드	설명
Version	포맷 버전. 로드 시 1 이 아니면 실패 반환 — 포맷 변경 대비 안전장치
NextUUID	다음에 발급할 UUID. 로드 후 신규 오브젝트가 기존 UUID와 충돌하지 않도록 복원
Primitives	UUID 문자열을 키로 하는 오브젝트 맵
Rotation	[Roll, Pitch, Yaw] 순서 (FRotator 멤버 순서와 일치)
Type	"Sphere" "Cube" "Cone" "Plane" "Camera" "Other"

11.2 저장 흐름

- 모든 액터를 순회하며 `GetRootComponent()` 를 얻는다(없으면 스킵)
- 루트의 `FTransform` 을 Location/Rotation/Scale 배열로 기록
- `Cast<UPrimitiveComponent>` 성공 시 `PrimitiveTypeToString` 으로 타입 문자열 기록,
`Cast<UCameraComponent>` 면 "Camera", 둘 다 아니면 "Other"
- 컴포넌트 UUID를 키로 `Primitives` 에 삽입
- `Json.dump(4)` 로 들여쓰기 포함 출력

11.3 불러오기 흐름

- 파일 열기 실패 / JSON 파싱 예외 / 버전 불일치는 각각 `false` 반환

2. NextUUID 복원

3. 각 엔트리에 대해 트랜스폼 복원 후 타입 분기

- "Camera" — 카메라 액터를 새로 스폰해 메인 카메라로 지정하고 트랜스폼 적용
- "Other" — 스킵
- 그 외 — AActor 스폰 후 AddPrimitiveComponent(Type, Transform), 저장된 UUID 복원

알려진 제약 — (1) 액터당 루트 컴포넌트 하나만 저장되므로 다중 컴포넌트 계층은 보존되지 않습니다. (2) Actor->SetUUID(UUID) 로 액터에 UUID를 복원하지만 저장 시 키는 컴포넌트의 UUID였으므로 두 값이 어긋납니다. 저장·복원 대상을 컴포넌트로 통일하는 편이 일관됩니다. (3) 색상·머티리얼·가시성(bVisible) 등은 아직 직렬화되지 않습니다 — UClass::GetProperties() 를 이용한 프로퍼티 기반 자동 직렬화로 확장할 여지가 있습니다.

12. 프레임 루프 전체 흐름

12.1 초기화 순서 (`Engine::Init`)

1. `FEditorUI` 생성 → `ConsolePanel` 등록 (이후 `LOG` 사용 가능하게 먼저)
2. Window 생성 (1280×720)
3. `FRenderer::Init` (디바이스·스왑체인·상태 객체)
4. `FResourceManager::Init(Renderer)`
5. `FImGuiRenderer::Init(hWnd, Device, Context)`
6. `GridRenderer` / `GizmoRenderer` / `OutlineRenderer` / `FGizmo` / `FOutline` 생성
7. `PropertyPanel` · `ControlPanel` 등록 + 씬 클리어 콜백 연결
8. `UWorld` 생성 → `World->Init()` (내부에서 카메라 액터 스폰)
9. 기본 큐브 액터 스폰 + 메시/셰이더 생성 후 바인딩
10. `bIsRunning = true`

콘솔 패널이 가장 먼저인 이유 — `LOG` 매크로가 `Engine::GetConsolePanel()` 을 참조하므로, 패널이 없으면 초기화 과정의 첫 로그에서 널 포인터 역참조가 발생합니다.

12.2 매 프레임 (Engine::Run)

```
while (bIsRunning)
{
    — 1. 시간 —————
    EngineTimer::Tick(); DeltaTime = EngineTimer::GetDeltaTime();

    — 2. 플랫폼 —————
    MainWindow->ProcessMessage(bIsRunning);          // 메시지 펌프 → 입력 상태 갱신
    if (MainWindow->CheckResized())                  // 리사이즈 반영
    {
        Renderer->Resize(W, H);
        Camera->SetAspectRatio((float)W / H);
    }

    — 3. 로직 갱신 —————
    World->Tick(DeltaTime);                          // BeginPlay 소진 → 액터 Tick
    EditorUI->Tick(DeltaTime);

    — 4. 조작 (기즈모 / 피킹) —————
    VP = Camera->GetViewProjectionMatrix();
    ray = Camera->DeProjection(MouseX, MouseY, W, H);
    Gizmo->Update(ray, mousePos, VP, W, H, bMouseDown, Camera);

    if (IsMousePressed(Left) && !Gizmo->IsUsing()
        && Gizmo->GetHoveredAxis() < 0 && !ImGui::GetIO().WantCaptureMouse)
    {
        Picked = World->GetPickingPrimitive(W, H);
        Gizmo->SetTarget(Picked); Outline->SetTarget(Picked);
        PropertyPanel->SetTarget(Picked);
    }

    — 5. 렌더 데이터 수집 —————
    TQueue<FRenderPacket> RenderQueue;
    World->GatherRenderPackets(RenderQueue);
    FInputSystem::UpdateInputStates();                // 입력 질의가 모두 끝난 뒤

    — 6. 렌더링 —————
    Renderer->BeginFrame();                          // RTV/DSV 클리어
    Renderer->BindShader(Shader);
    GridRenderer->OnRender(VP, CameraLocation);
    Renderer->RenderAll(RenderQueue, VP);              // 씬 오브젝트

    if (Outline->GetTarget()) OutlineRenderer->OnRender(*Outline, VP);
    if (Gizmo->GetTarget())
    {
        Renderer->SetDepthStencilEnabled(false);    // 기즈모는 항상 위에
        GizmoRenderer->OnRender(*Gizmo, VP);
    }

    — 7. UI —————
    ImGuiRenderer->Begin();
    EditorUI->OnRender();
    ImGuiRenderer->End();
}
```

```
Renderer->EndFrame(); // Present
}
```

피킹 조건이 세 겹인 이유

- `!Gizmo->IsUsing()` — 기즈모를 드래그 중이면 선택이 바뀌지 않도록
- `Gizmo->GetHoveredAxis() < 0` — 기즈모 축 위를 클릭한 경우 뒤쪽 오브젝트가 선택되지 않도록
- `!ImGui::GetIO().WantCaptureMouse` — ImGui 창 위 클릭이 씬으로 새지 않도록

12.3 종료 (`Engine::Shutdown`)

```
delete World; // ~UWorld 가 모든 액터 delete
ImGuiRenderer->Shutdown();
Renderer->Shutdown(); // ComPtr 해제 → D3D 리소스 반환
```

`Engine.cpp` 상단에 전역 `void operator delete(void* Ptr) {}` 가 빈 구현으로 정의되어 있습니다. 프로그램 전역의 모든 `delete` 가 아무 일도 하지 않게 되어 **모든 힙 메모리가 해제되지 않습니다**. `UObject` 의 클래스 스코프 `operator delete` 와 별개이며, 디버깅용 임시 코드로 보입니다. 정리 시 제거를 권합니다.

13. 확장 가이드

13.1 새 프리미티브 타입 추가하기

1. `Component/PrimitiveComponent.h` 의 `EPrimitiveType` 에 항목 추가
2. `Render/GeometryGenerator.h/.cpp` 에 `CreateXxx()` 생성 함수 작성 — `FMeshData` (정점 + 인덱스) 반환
3. `FGeometryGenerator::GetMeshData(Name)` 및 `FResourceManager::GetMesh` 분기에 이름 매핑 추가
4. `World.cpp` 의 `PrimitiveTypeToString` / `FStringToPrimitiveType` 양쪽에 문자열 추가 (씬 저장·복원)
5. `Editor/ControlPanel.h` 의 `Items[]` 배열과 크기를 갱신해 스폰 콤보박스에 노출

`Items[4]` 처럼 크기가 하드코딩되어 있으므로 항목을 늘릴 때 배열 크기와 `IM_ARRAYSIZE` 사용 여부를 함께 확인해야 합니다. 이 다섯 곳이 흩어져 있는 만큼, 타입 하나를 { enum, 문자열, 생성 함수 } 테이블로 묶어두면 추가 지점이 한 곳으로 줄어듭니다.

13.2 새 컴포넌트 클래스 추가하기

```
#pragma once
#include "SceneComponent.h"

class ULightComponent : public USceneComponent
{
    DECLARE_CLASS(ULightComponent, USceneComponent)

    REFLECT_START(ClassName)
        PROPERTY(Intensity)          // float - Property.h에 이미 등록된 타입
        PROPERTY(bCastShadow)        // bool
    REFLECT_END()
public:
    virtual void BeginPlay() override;
    virtual void TickComponent(float DeltaTime) override;
private:
    float Intensity    = 1.0f;
    bool bCastShadow = true;
};
```

주의할 점: `PROPERTY` 는 `offsetof` 를 사용하므로 대상 멤버가 표준 레이아웃 위치에 있어야 하고, 새 타입을 쓰려면 `Property.h` 에 `DEFINE_PROPERTY_TYPE` 을 추가해야 합니다. 인스턴스 생성은 항상 `FObjectFactory::ConstructObject<ULightComponent>()` 로 합니다.

13.3 새 에디터 패널 추가하기

```
class FStatsPanel : public IEditorPanel
{
public:
    bool Init() override { return true; }
    void Tick(float Dt) override { /* 상태 집계 */ }
    void OnRender() override
    {
        ImGui::Begin("Stats");
        ImGui::Text("Actors: %d", World->GetActorNum());
        ImGui::Text("Alloc : %llu bytes", FEngineStatics::TotalAllocationBytes);
        ImGui::End();
    }
    UWorld* World = nullptr;
};

// Engine::Init 안에서
FStatsPanel* Stats = EditorUI->AddEditorPanel<FStatsPanel>();
Stats->World = World;
```

13.4 새 셰이더 패스 추가하기

1. Shader/ 에 .hlsl 작성 (진입점 mainVS / mainPS 컨벤션 확인)
2. Renderer->CreateShader(L"Shader/Xxx.hlsl", Layout) 로 FShader 생성 — 가능하면 FVertex::GetLayout() 재사용
3. 필요한 상수 버퍼를 CreateConstantBuffer(sizeof(FMyConstants)) 로 만들고 슬롯 번호를 정함
4. 렌더 시 UpdateConstantBufferData → BindConstantBuffer(Slot, CB, Vertex|Pixel) → BindShader → BindMesh → DrawIndexed
5. 깊이 처리가 다른면 SetDepthStencilEnabled 로 상태 전환 (기즈모 렌더 방식 참고)

13.5 개선 후보 정리

항목	현재	제안
전역 operator delete	빈 구현 — 메모리 해제 안 됨	제거
정점 레이아웃	오프셋 12 가 두 곳에 하드코딩	offsetof + FVertex::GetLayout() 으로 일원화
피킹 t 비교	로컬 공간 t 를 오브젝트 간 직접 비교	월드 거리로 환산 후 비교, 역행렬 캐시
MeshData 사본	컴포넌트마다 정점 배열 복제	TSharedPtr<FMeshData> 로 공유
씬 직렬화	Transform + Type만 저장	UClass::GetProperties() 기반 자동 직렬화
FOutline::GetWorldMatrix	임시 객체에 대한 참조 반환	값 반환으로 변경
렌더 큐	정렬·컬링 없음	불투명/반투명 분리, 프러스텀 컬링, 인스턴싱
ImGui::ShowDemoWindow()	메인 루프에 남아 있음	디버그 플래그로 토글

